DPOO-2526

NUDE EYE PROJECT

DPOO-P1-Report-Group 1

Jan Francesc Porcar Guillen - janfrancesc.porcar

Francesc Mateu Vallespín - f.mateu

Table of Contents

Summary.....1
Class Diagram.....2
Key Design Decisions.....11
Dedication Estimate.....14
Conclusions.....17
Bibliography.....19

Summary

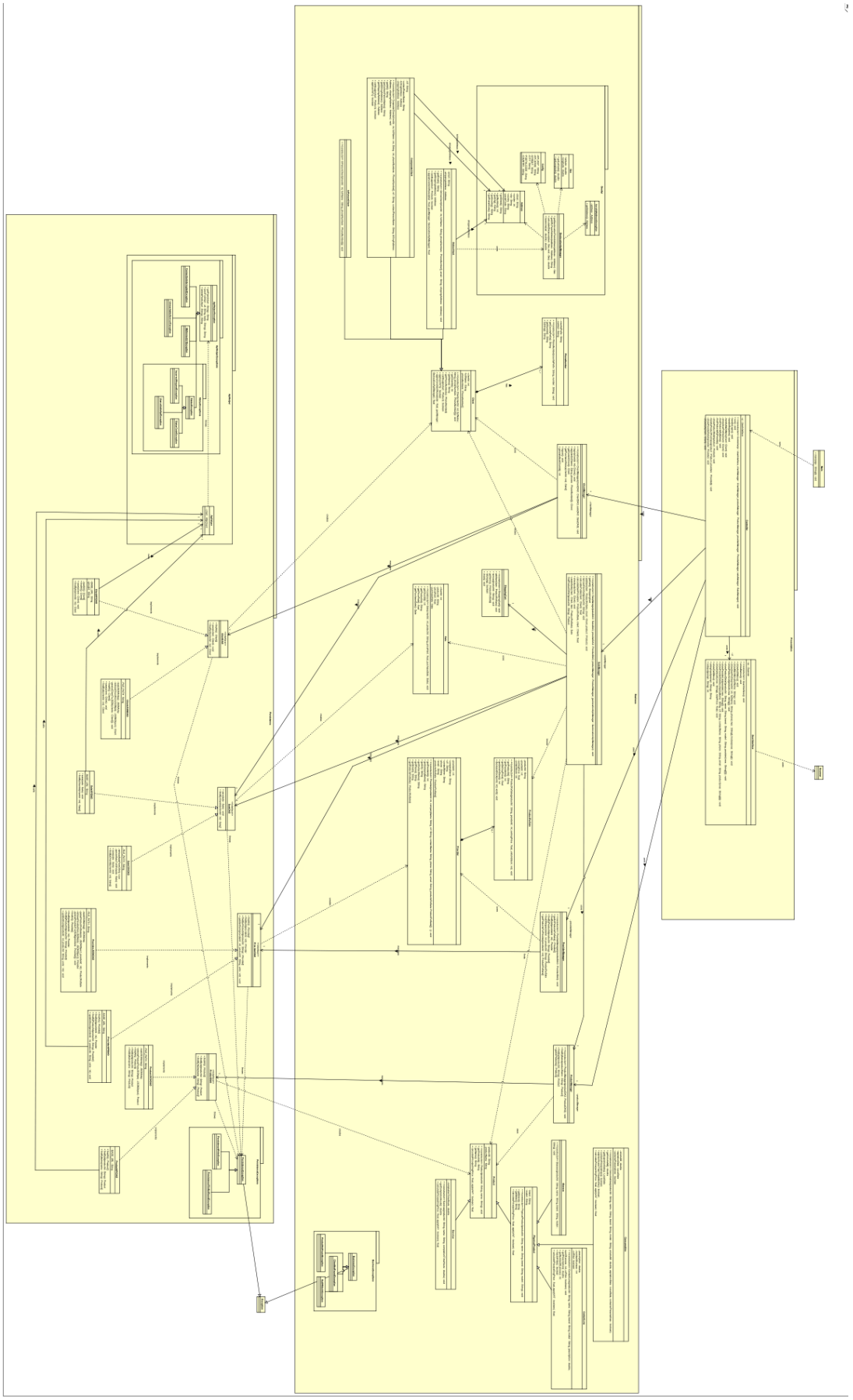
The Nude Eye application is a Java console program designed to manage the operations of an optical shop. Users can register or log in, browse a product catalogue, build a shopping cart, and complete purchases. The system stores a full purchase history per client, accessible from the user profile at any time.

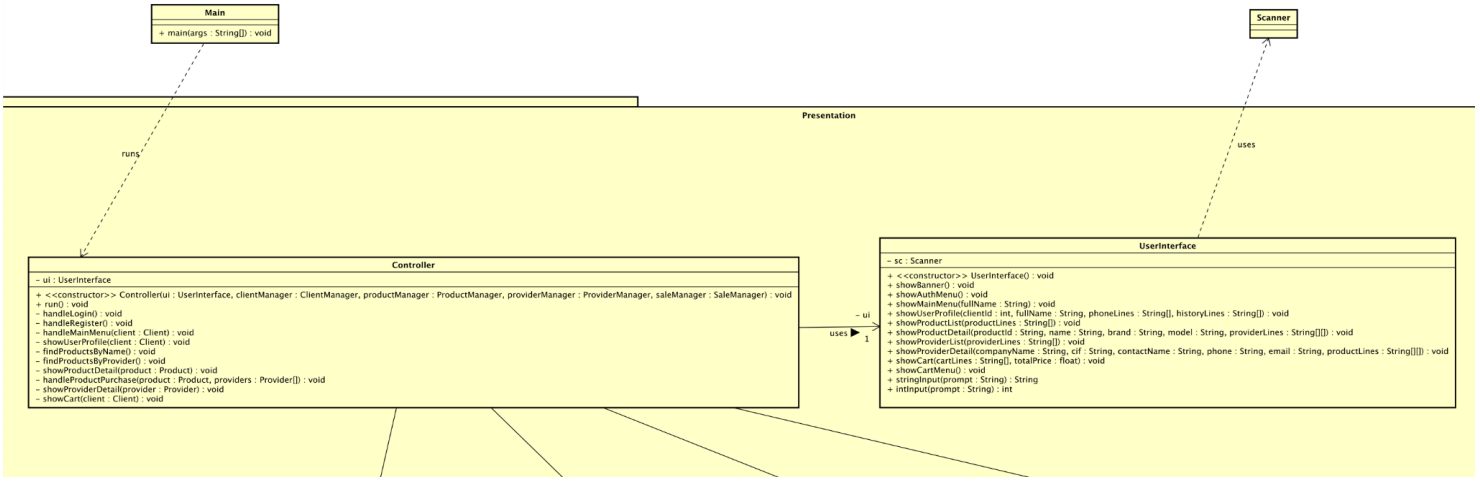
Phase 3–4 introduced two major extensions to the existing codebase. The first is a specialised client and product model built on inheritance. The second is a dual-persistence mechanism that selects between a remote REST API and local files depending on availability at startup.

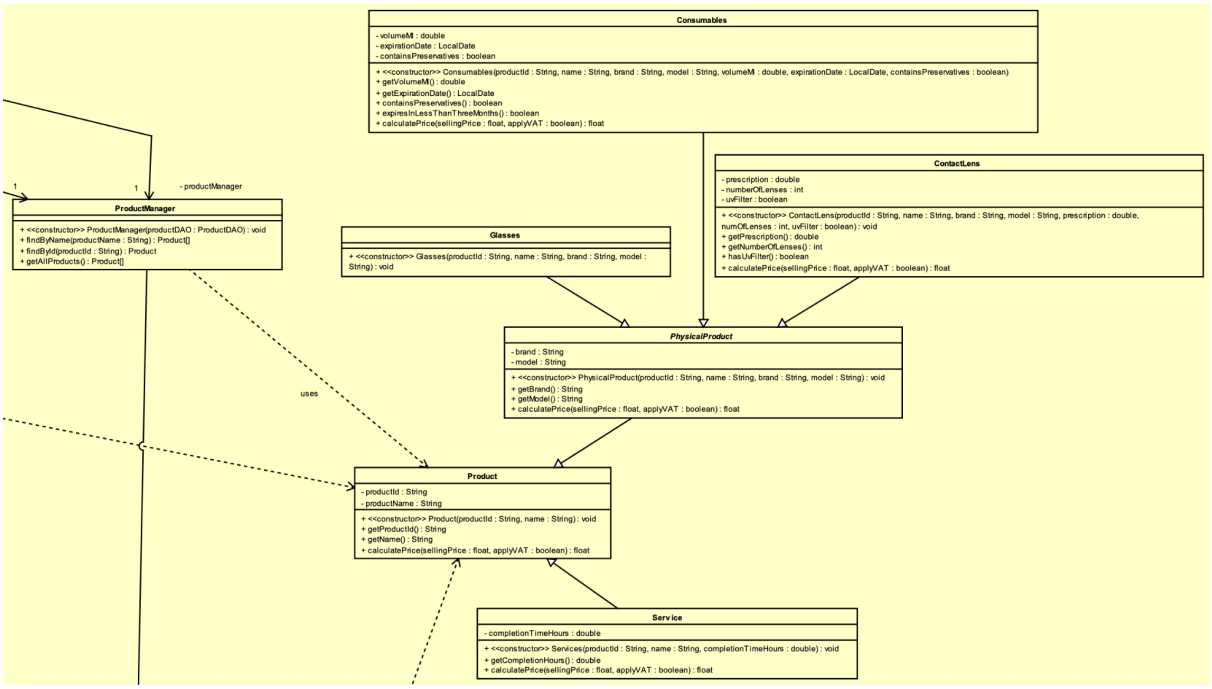
On the domain side, Client and Product are now abstract classes with concrete subtypes that each carry their own behaviour. Three client types exist: InPersonClient, which has no additional attributes beyond the base class; OnlineClient, which stores an email and a shipping address and adds a distance-based shipping fee at checkout; and CorporateClient, which stores a NIF, a contact person, and two addresses, and is exempt from VAT when its billing country is not Spain. Four product types exist: Glasses and ContactLens both extend an intermediate PhysicalProduct class that holds brand and model; Consumable adds volume, expiration date, and preservative information; and Service holds a completion time in hours and extends Product directly since it has no physical attributes. Each subtype implements its own price calculation: contact lenses apply 10% VAT, consumables apply 21% VAT with a 40% markdown when close to expiry, and services multiply the base price by duration before applying VAT.

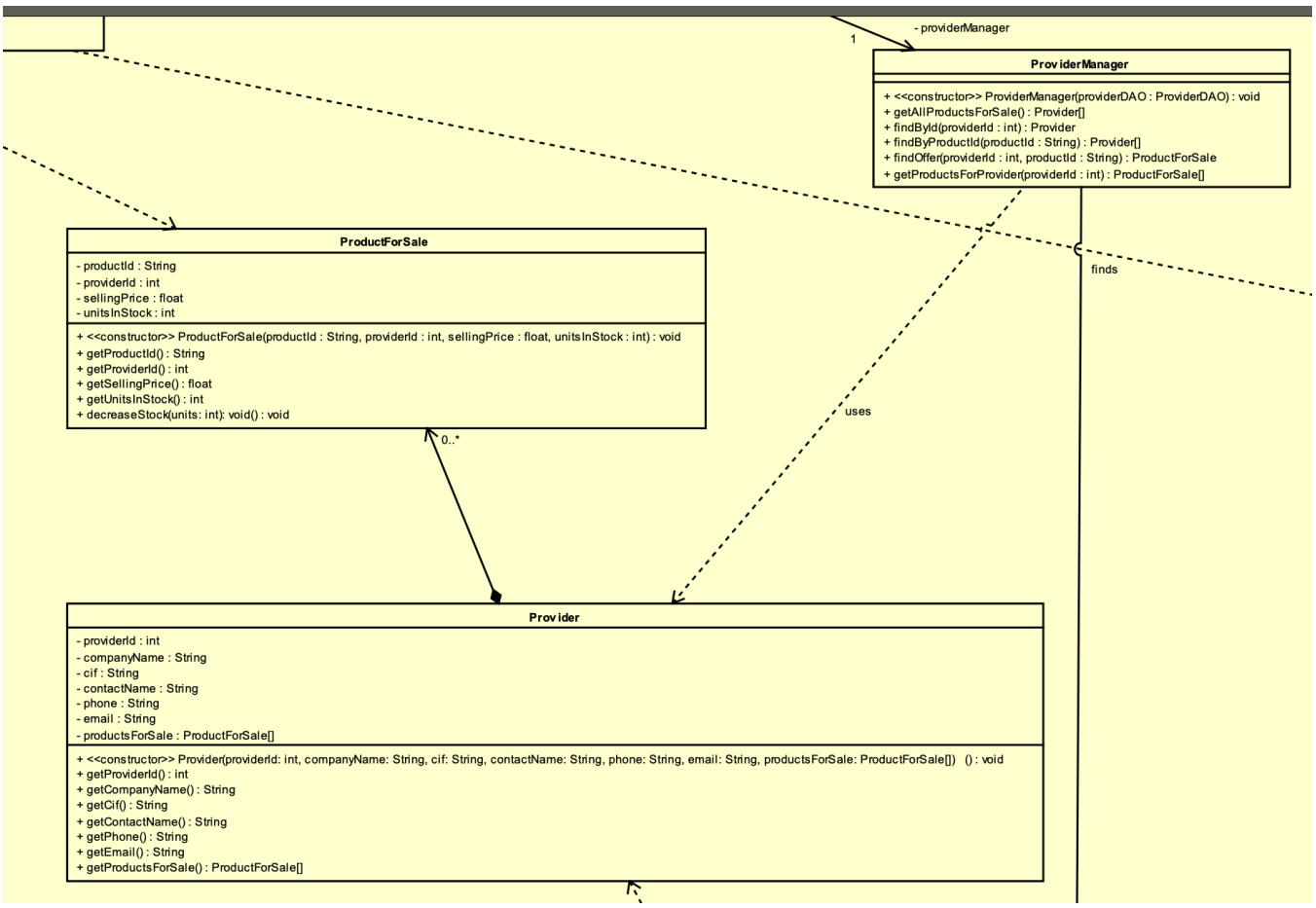
Persistence is handled through DAO interfaces, one per entity, with two implementations each. The Controller probes the API at launch and injects either the API-based or the file-based DAOs into the managers. If neither source is reachable, the program halts rather than operating on missing data.

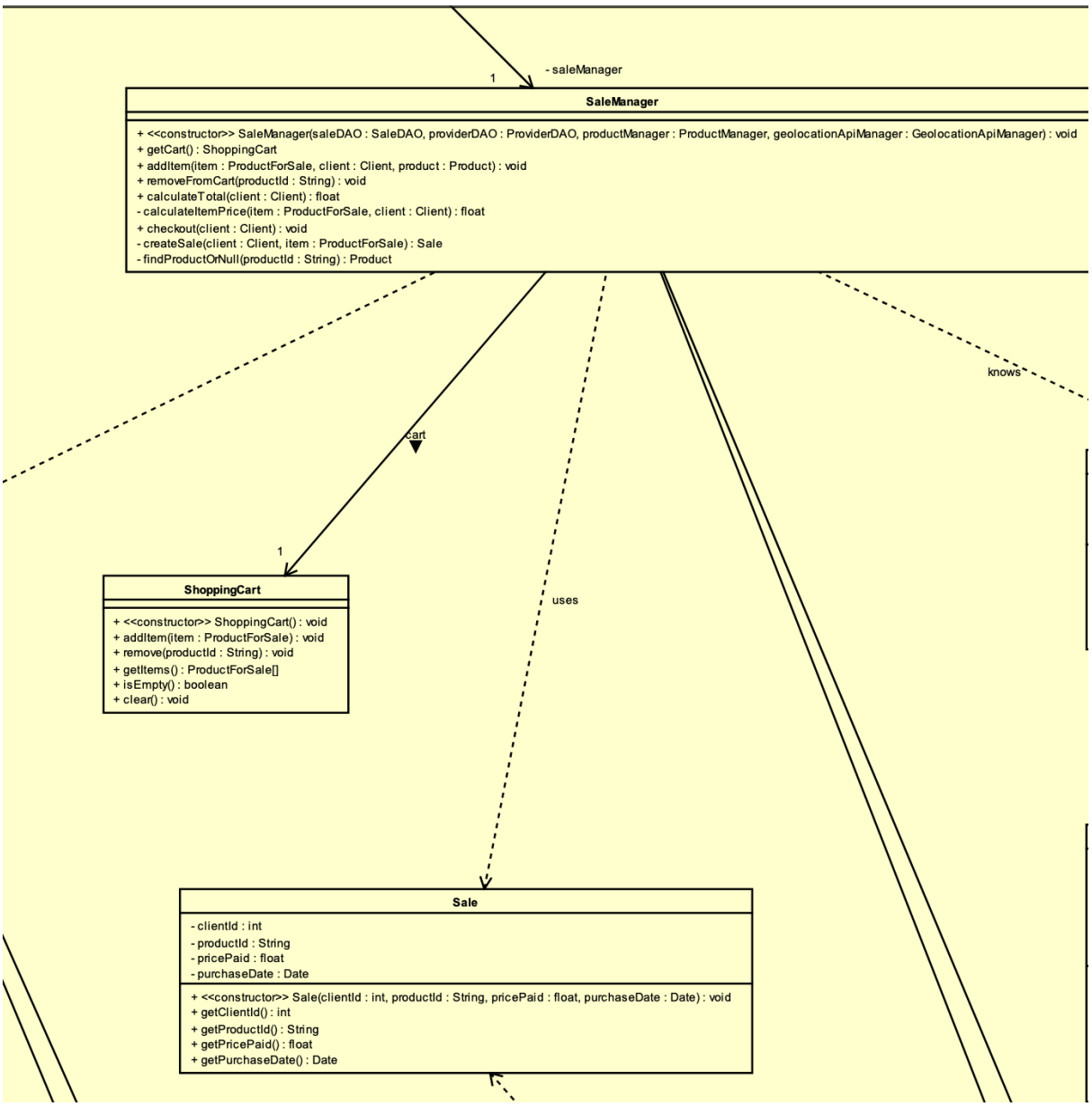
Class Diagram

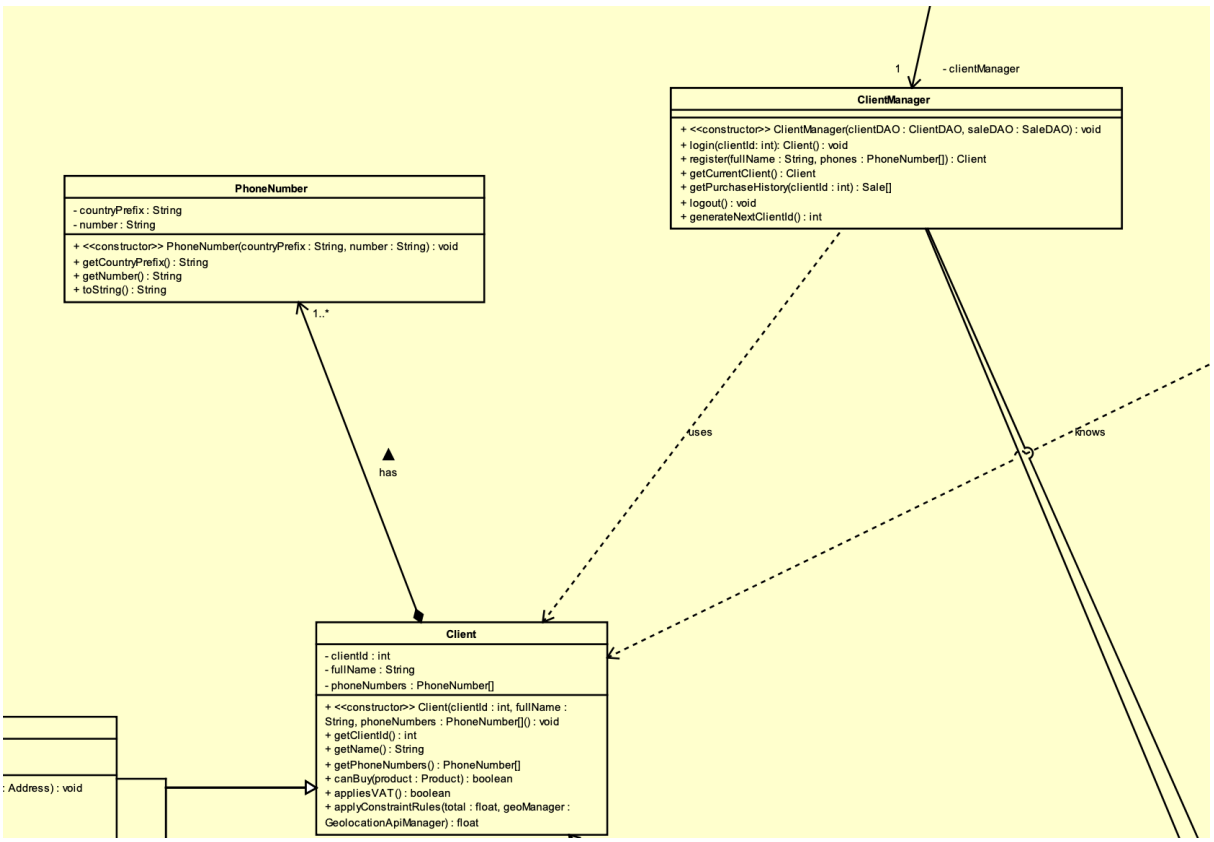


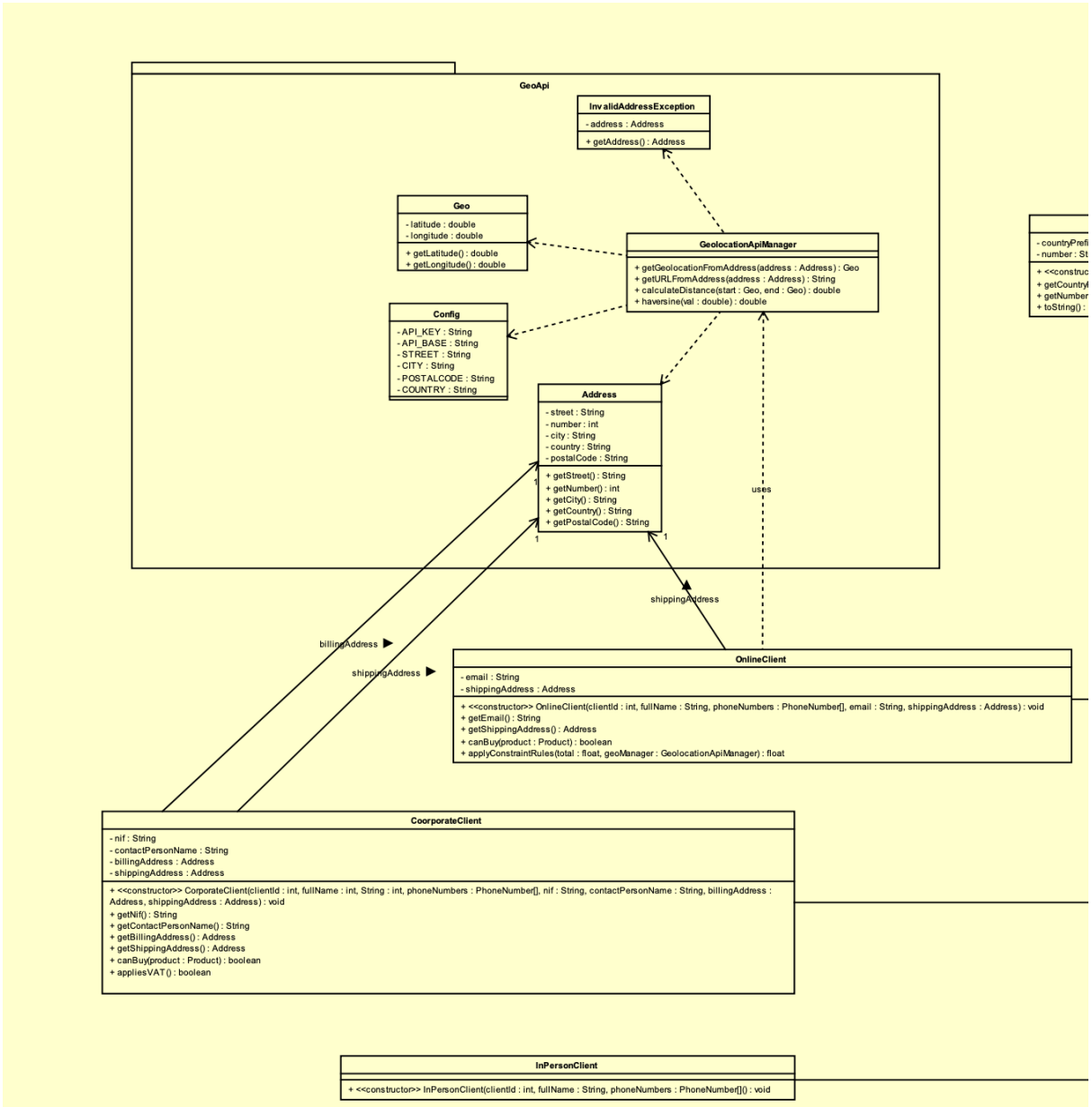

Main
+ main(args : String[]) : void
Scanner
Presentation
runs
uses
Controller
- ui : UserInterface
+ <<constructor>> Controller(ui : UserInterface, clientManager : ClientManager, productManager : ProductManager, providerManager : ProviderManager, saleManager : SaleManager) : void
+ run() : void
- handleLogin() : void
- handleRegister() : void
- handleMainMenu(client : Client) : void
- showUserProfile(client : Client) : void
- findProductsByName() : void
- findProductsByProvider() : void
- showProductDetail(product : Product) : void
- handleProductPurchase(product : Product, providers : Provider[]) : void
- showProviderDetail(provider : Provider) : void
- showCart(client : Client) : void
- ui
uses
1
UserInterface
- sc : Scanner
+ <<constructor>> UserInterface() : void
+ showBanner() : void
+ showAuthMenu() : void
+ showMainMenu(fullName : String) : void
+ showUserProfile(clientId : int, fullName : String, phoneLines : String[], historyLines : String[]) : void
+ showProductList(productLines : String[]) : void
+ showProductDetail(productId : String, name : String, brand : String, model : String, providerLines : String[][]) : void
+ showProviderList(providerLines : String[]) : void
+ showProviderDetail(companyName : String, cif : String, contactName : String, phone : String, email : String, productLines : String[][]) : void
+ showCart(cartLines : String[], totalPrice : float) : void
+ showCartMenu() : void
+ stringInput(prompt : String) : String
+ intInput(prompt : String) : int

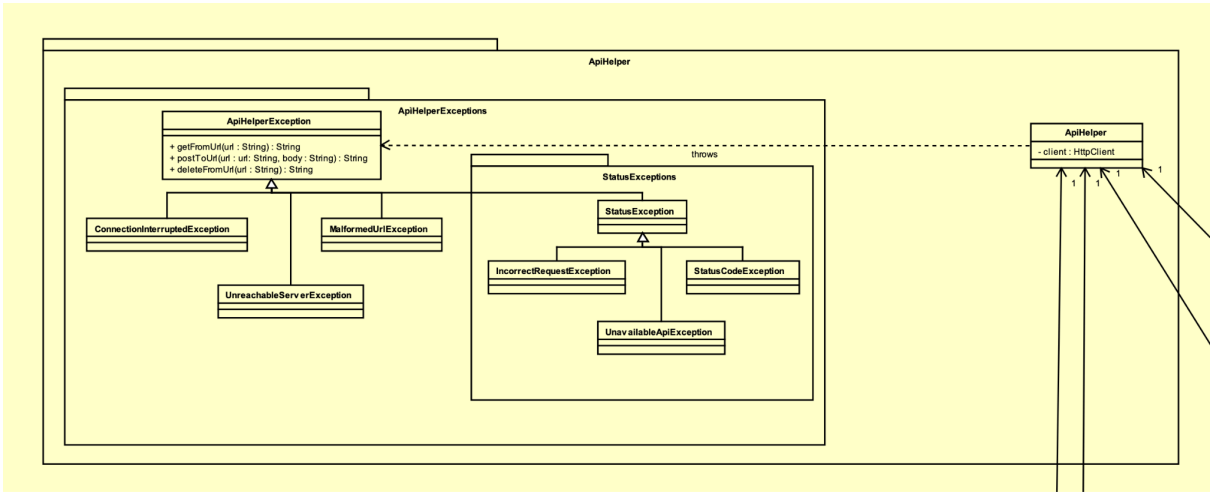

Consumables
- volumeM : double
- expirationDate : LocalDate
- containsPreservatives : boolean
+ <<constructor>> Consumables(productId : String, name : String, brand : String, model : String, volumeM : double, expirationDate : LocalDate, containsPreservatives : boolean)
+ getVolumeM() : double
+ getExpirationDate() : LocalDate
+ containsPreservatives() : boolean
+ expiresInLessThanThreeMonths() : boolean
+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float
1
1
- productManager
ProductManager
+ <<constructor>> ProductManager(productDAO : ProductDAO) : void
+ findByName(productName : String) : Product[]
+ findById(productId : String) : Product
+ getAllProducts() : Product[]
Glasses
+ <<constructor>> Glasses(productId : String, name : String, brand : String, model : String) : void
ContactLens
- prescription : double
- numberOfLenses : int
- uvFilter : boolean
+ <<constructor>> ContactLens(productId : String, name : String, brand : String, model : String, prescription : double, numOfLenses : int, uvFilter: boolean) : void
+ getPrescription() : double
+ getNumberOfLenses() : int
+ hasUvFilter() : boolean
+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float
PhysicalProduct
- brand : String
- model : String
+ <<constructor>> PhysicalProduct(productId : String, name : String, brand : String, model : String) : void
+ getBrand() : String
+ getModel() : String
+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float
uses
Product
- productId : String
- productName : String
+ <<constructor>> Product(productId : String, name : String) : void
+ getProductId() : String
+ getName() : String
+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float
Service
- completionTimeHours : double
+ <<constructor>> Services(productId : String, name : String, completionTimeHours : double) : void
+ getCompletionHours() : double
+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float

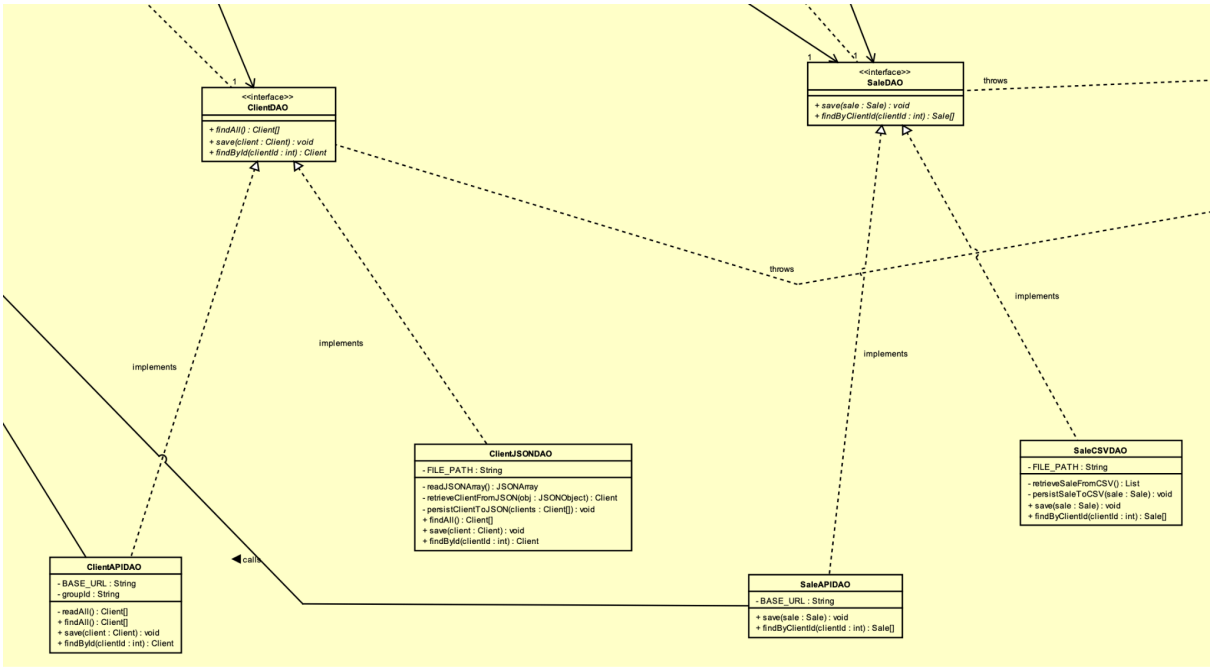

- providerManager
1
ProviderManager
+ <<constructor>> ProviderManager(providerDAO : ProviderDAO) : void
+ getAllProductsForSale() : Provider[]
+ findById(providerId : int) : Provider
+ findByProductId(productId : String) : Provider[]
+ findOffer(providerId : int, productId : String) : ProductForSale
+ getProductsForProvider(providerId : int) : ProductForSale[]
finds
ProductForSale
- productId : String
- providerId : int
- sellingPrice : float
- unitsInStock : int
+ <<constructor>> ProductForSale(productId : String, providerId : int, sellingPrice : float, unitsInStock : int) : void
+ getProductId() : String
+ getProviderId() : int
+ getSellingPrice() : float
+ getUnitsInStock() : int
+ decreaseStock(units: int): void() : void
uses
0..*
Provider
- providerId : int
- companyName : String
- cif : String
- contactName : String
- phone : String
- email : String
- productsForSale : ProductForSale[]
+ <<constructor>> Provider(providerId: int, companyName: String, cif: String, contactName: String, phone: String, email: String, productsForSale: ProductForSale[]) () : void
+ getProviderId() : int
+ getCompanyName() : String
+ getCif() : String
+ getContactName() : String
+ getPhone() : String
+ getEmail() : String
+ getProductsForSale() : ProductForSale[]

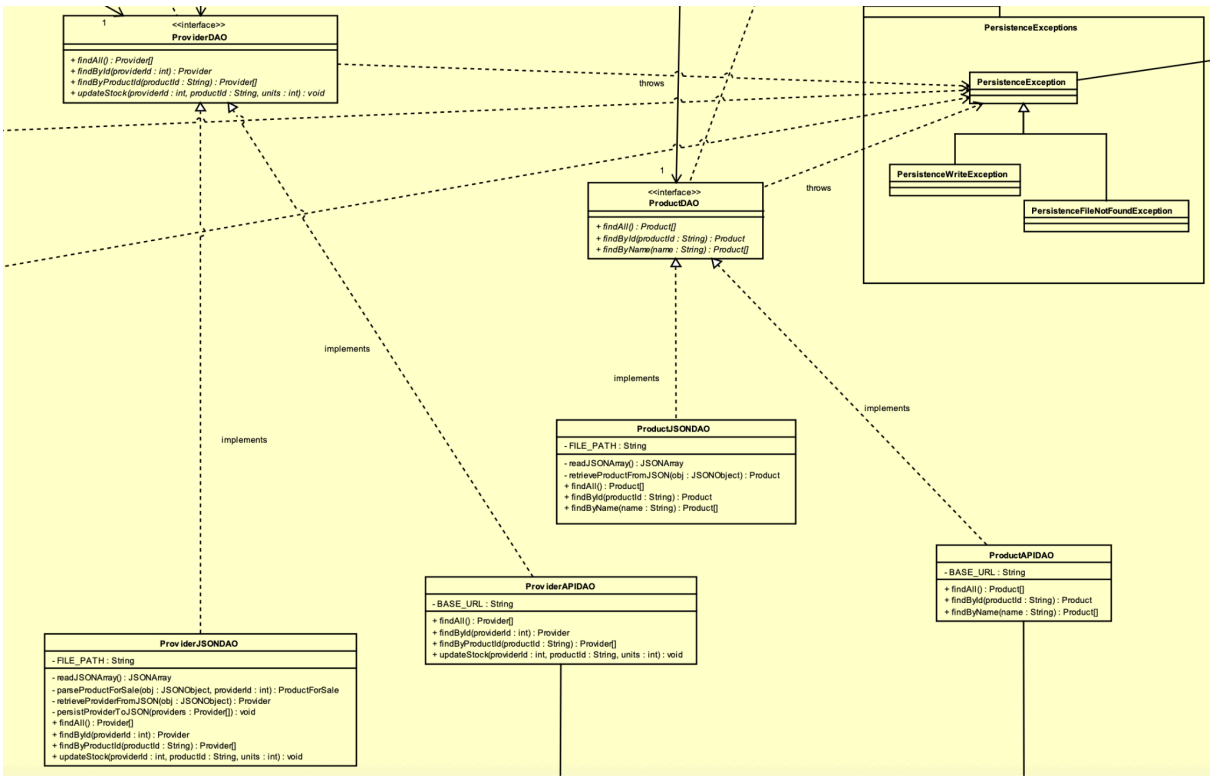

- saleManager
1
SaleManager
+ <<constructor>> SaleManager(saleDAO : SaleDAO, providerDAO : ProviderDAO, productManager : ProductManager, geolocationApiManager : GeolocationApiManager) : void
+ getCart() : ShoppingCart
+ addItem(item : ProductForSale, client : Client, product : Product) : void
+ removeFromCart(productId : String) : void
+ calculateTotal(client : Client) : float
- calculateItemPrice(item : ProductForSale, client : Client) : float
+ checkout(client : Client) : void
- createSale(client : Client, item : ProductForSale) : Sale
- findProductOrNull(productId : String) : Product
knows
cart
1
ShoppingCart
+ <<constructor>> ShoppingCart() : void
+ addItem(item : ProductForSale) : void
+ remove(productId : String) : void
+ getItems() : ProductForSale[]
+ isEmpty() : boolean
+ clear() : void
uses
Sale
- clientId : int
- productId : String
- pricePaid : float
- purchaseDate : Date
+ <<constructor>> Sale(clientId : int, productId : String, pricePaid : float, purchaseDate : Date) : void
+ getClientId() : int
+ getProductId() : String
+ getPricePaid() : float
+ getPurchaseDate() : Date


1
- clientManager
ClientManager
+ <<constructor>> ClientManager(clientDAO : ClientDAO, saleDAO : SaleDAO) : void
+ login(clientId: int): Client() : void
+ register(fullName : String, phones : PhoneNumber[]) : Client
+ getCurrentClient() : Client
+ getPurchaseHistory(clientId : int) : Sale[]
+ logout() : void
+ generateNextClientId() : int
PhoneNumber
- countryPrefix : String
- number : String
+ <<constructor>> PhoneNumber(countryPrefix : String, number : String) : void
+ getCountryPrefix() : String
+ getNumber() : String
+ toString() : String
1..*
has
uses
knows
Client
- clientId : int
- fullName : String
- phoneNumbers : PhoneNumber[]
+ <<constructor>> Client(clientId : int, fullName : String, phoneNumbers : PhoneNumber[]() : void
+ getClientId() : int
+ getName() : String
+ getPhoneNumbers() : PhoneNumber[]
+ canBuy(product : Product) : boolean
+ appliesVAT() : boolean
+ applyConstraintRules(total : float, geoManager : GeolocationApiManager) : float
: Address) : void


GeoApi
InvalidAddressException
- address : Address
+ getAddress() : Address
Geo
- latitude : double
- longitude : double
+ getLatitude() : double
+ getLongitude() : double
GeolocationApiManager
+ getGeolocationFromAddress(address : Address) : Geo
+ getURLFromAddress(address : Address) : String
+ calculateDistance(start : Geo, end : Geo) : double
+ haversine(val : double) : double
Config
- API_KEY : String
- API_BASE : String
- STREET : String
- CITY : String
- POSTALCODE : String
- COUNTRY : String
Address
- street : String
- number : int
- city : String
- country : String
- postalCode : String
+ getStreet() : String
+ getNumber() : int
+ getCity() : String
+ getCountry() : String
+ getPostalCode() : String
uses
shippingAddress
billingAddress
shippingAddress
OnlineClient
- email : String
- shippingAddress : Address
+ <<constructor>> OnlineClient(clientId : int, fullName : String, phoneNumbers : PhoneNumber[], email : String, shippingAddress : Address) : void
+ getEmail() : String
+ getShippingAddress() : Address
+ canBuy(product : Product) : boolean
+ applyConstraintRules(total : float, geoManager : GeolocationApiManager) : float
CoorporateClient
- nif : String
- contactPersonName : String
- billingAddress : Address
- shippingAddress : Address
+ <<constructor>> CorporateClient(clientId : int, fullName : int, String : int, phoneNumbers : PhoneNumber[], nif : String, contactPersonName : String, billingAddress : Address, shippingAddress : Address) : void
+ getNif() : String
+ getContactPersonName() : String
+ getBillingAddress() : Address
+ getShippingAddress() : Address
+ canBuy(product : Product) : boolean
+ appliesVAT() : boolean
InPersonClient
+ <<constructor>> InPersonClient(clientId : int, fullName : String, phoneNumbers : PhoneNumber[]() : void


ApiHelper
ApiHelperExceptions
ApiHelperException
+ getFromUrl(url : String) : String
+ postToUrl(url : url: String, body : String) : String
+ deleteFromUrl(url : String) : String
throws
ApiHelper
- client : HttpClient
StatusExceptions
StatusException
ConnectionInterruptedException
MalformedUrlException
IncorrectRequestException
StatusCodeException
UnreachableServerException
UnavailableApiException


<<interface>>
ClientDAO
+ findAll() : Client[]
+ save(client : Client) : void
+ findById(clientId : int) : Client
<<interface>>
SaleDAO
+ save(sale : Sale) : void
+ findByClientId(clientId : int) : Sale[]
throws
throws
implements
implements
implements
implements
ClientJSONDAO
- FILE_PATH : String
- readJSONArray() : JSONArray
- retrieveClientFromJSON(obj : JSONObject) : Client
- persistClientToJSON(clients : Client[]) : void
+ findAll() : Client[]
+ save(client : Client) : void
+ findById(clientId : int) : Client
SaleCSVDAO
- FILE_PATH : String
- retrieveSaleFromCSV() : List
- persistSaleToCSV(sale : Sale) : void
+ save(sale : Sale) : void
+ findByClientId(clientId : int) : Sale[]
ClientAPIDAO
- BASE_URL : String
- groupId : String
- readAll() : Client[]
+ findAll() : Client[]
+ save(client : Client) : void
+ findById(clientId : int) : Client
SaleAPIDAO
- BASE_URL : String
+ save(sale : Sale) : void
+ findByClientId(clientId : int) : Sale[]



One of the important decisions included in the class diagram concerns the division of the program according to responsibility layers. The Presentation layer includes the classes Controller and UserInterface. The Controller is responsible for managing the process flow of the program, while the only class that communicates with the user through the console is the UserInterface. This makes sure that the responsibility of reading data and displaying results is separated from the business classes, making the program independent from any presentation details.

The Business layer is the main layer of the application that consists of the model and manager classes. Managers provide for the realization of the use cases of the application. The four main use cases include clients, products, providers, and sales. Managers perform coordination of the business process, but all specific actions are carried out by the entity objects. For instance, SaleManager performs coordination of the checkout procedure, while the actual calculation of product price and conditions for sale is performed by the concrete subclass objects.

Finally, the model also uses inheritance for the client and product classes. The Client class is used as a superclass for the InPersonClient, OnlineClient, and CorporateClient subclasses, which store different types of information and have different behavior.

Specifically, OnlineClient class contains the attribute for storing shipping address, and OnlineClient class can calculate the cost taking into account the shipping fee. Also, CorporateClient class contains the attribute with fiscal info and does not pay VAT if the billing address country is not Spain. Abstract Product class serves as the superclass for PhysicalProduct, as a subclass that represents items with brand and model, and the Service class that is not tangible inherits from the Product class directly.

An important design decision that can be seen in this architecture is the use of DAO interfaces in the Persistence package. The interfaces ClientDAO, ProductDAO, ProviderDAO, and SaleDAO provide the list of actions performed on data, but not the way it is performed. Concrete implementations of DAO, including JSON/CSV DAOs and API DAOs, hide data access and API calling operations, thus making it easy to change the data storage from local storage to cloud one.

The choice of which runtime will be used, either API or local storage, takes place during application initialization. If the API is applicable for use, the Controller determines this and informs the managers about it. In turn, each Manager uses the DAO interface to create an implementation of the DAO corresponding to its particular case. By doing so, the Main class becomes a small piece of code responsible exclusively for starting the application, and there are no references to persistence implementations inside the Controller.

Moreover, API related classes remain outside the core part of the project. ApiHelper is only used by API implementations of DAOs as it is a technical aspect of remote persistence. Similarly, the GeolocationApiManager is put into the separate GeoApi package and referenced only when online shipping must be calculated. By keeping external aspects outside the main package structure, we simplify our design for better understanding in a UML diagram.

Ultimately, the exception model is illustrated because this solution utilizes exceptions to decouple the error-handling logic from the business process. Business exceptions signify errors within the business rules, for instance when the product is not available and there is no product in stock. On the other hand, persistence exceptions signify

issues with regard to files or the API. The decoupling will help to identify whether an error is within the business rules of the shop or within the technical architecture itself.

Key Design Decisions

The most critical design choice here was creating abstract Client and Product classes, which prevent from creating generic objects, making each client and each product be instantiated as some subtype, which is required for polymorphism. In order to not make the Service inherit properties, which would have been irrelevant, brand and model were shifted from Product to intermediate PhysicalProduct class, containing common structure of Glasses, ContactLens, and Consumable.

In order not to violate OCP rule by adding some code to SaleManager, VAT was implemented using a hook method appliesVAT(), created in abstract Client and returning true by default, but overridden by CorporateClient for returning false if the billing address belongs to any other country except Spain.

This way, SaleManager can simply call client.appliesVAT() and pass its return value as boolean argument in calculatePrice(float, boolean). Each Product subclass handles this independently, applying 10%, 21% (with 40% discount for expiring within three months), or multiplication by completion time, followed by VAT, respectively.

Online shipping is calculated per checkout transaction rather than per item purchased. The OnlineClient override method applyConstraintRules() computes the fee by adding €2.00 base and €0.01 for every kilometre between La Salle and the location indicated by the Client's geolocation, using the GeolocationApiManager API. Other types of Clients do nothing, leaving SaleManager free of shipping concerns.

All price-related constants (VAT rates, expiry discount, online shipping prices, and La Salle coordinates) are grouped into the final PricingConstants class. As a result, any modifications related to the above parameters will be made solely in that class.

The choice of DAO pattern implies one common interface per managed entity (ClientDAO, ProductDAO, ProviderDAO, SaleDAO) with two concrete

implementations (JSON/CSV file and an API) for local and cloud storages. At runtime, the Controller decides which implementation to use and then injects corresponding DAO to all managers, thus ensuring that the business layer does not depend on how objects are persisted.

Lastly, in order to properly deserialise a product instance from JSON payload, the discriminator field product_type is used by DAO to select the correct concrete class to instantiate.

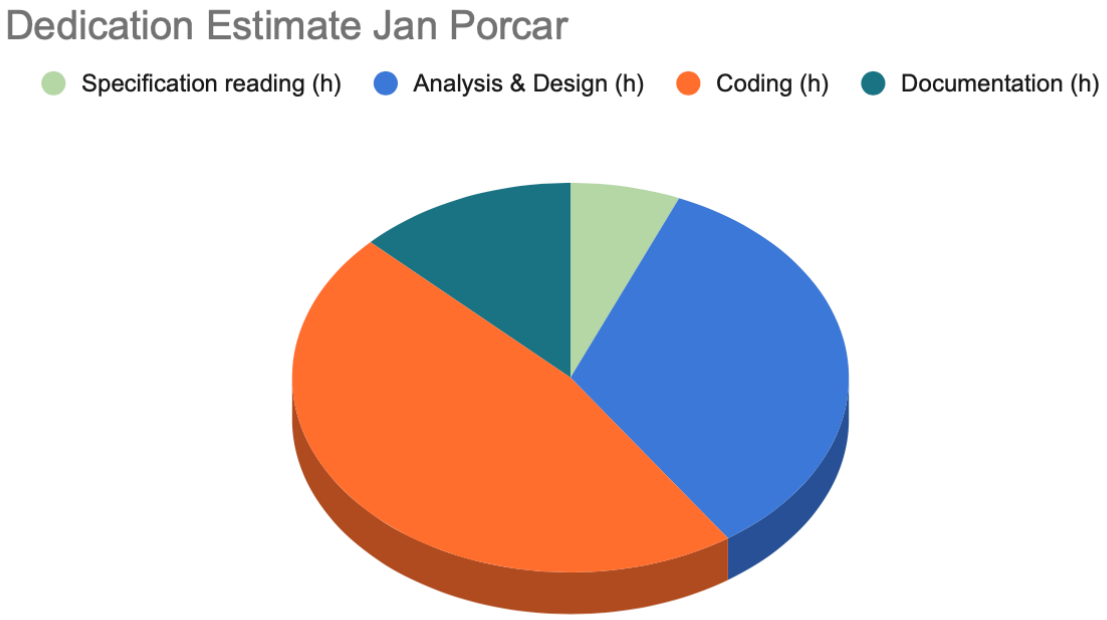
The same strategy is employed while dealing with the clients. As Client is an abstract class, it is not possible to create a generic client object using JSON or REST API call. That is why client data being serialized has information regarding the concrete client class. The DAO uses this value and recreates the appropriate subclass, either InPersonClient, OnlineClient, or CorporateClient. It has to be done because each sub-class has different fields and functionality; for instance, OnlineClient keeps information related to shipping and adds cost of delivery, whereas CorporateClient holds fiscal information and is eligible for exemption from VAT when billing country is not Spain.

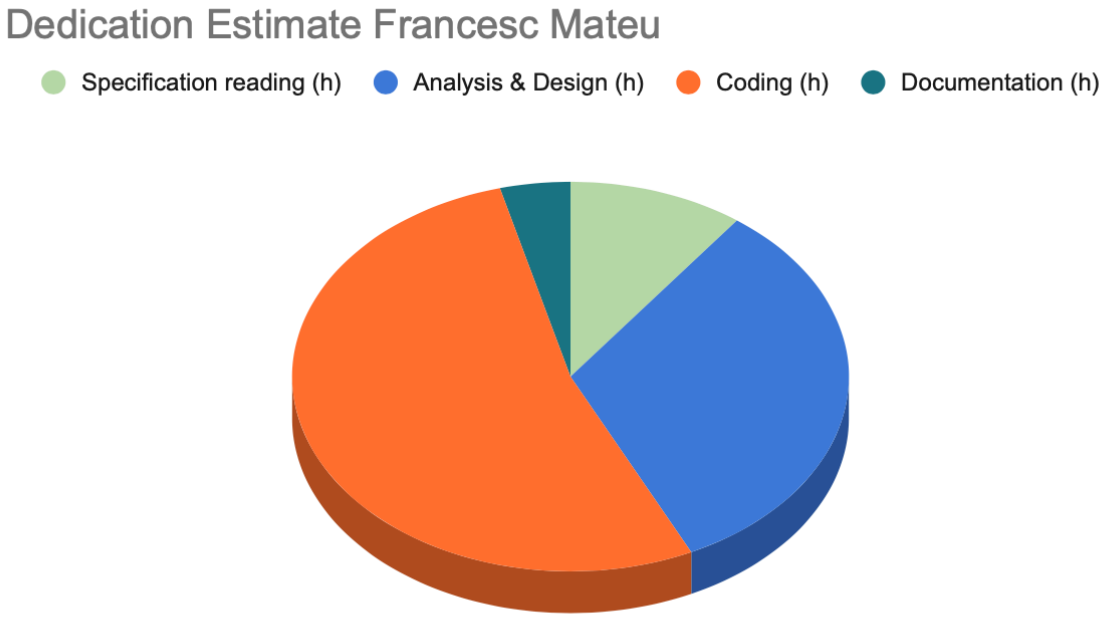
Also worth mentioning in terms of the decision made was keeping the Main class responsible only for starting the application. The Main class just sets up logging and initializes the Controller, after which the application starts. The choice between using API-based persistence and local file-based persistence is not delegated to Main class, because choosing persistence approach is part of the application logic itself and can depend on API presence. The Controller makes the decision and gets a Boolean parameter that tells whether to use API or not.

Then, the boolean is passed to each business manager when they are being initialized. Every manager picks its persistence implementation on its own internally. So, ProductManager works with ProductDAO interface, but when creating itself determines whether to instantiate ProductAPIDAO or ProductJSONDAO based on the value provided by the controller. In this way, Controller does not have any connection to particular persistence implementations, but no additional initializing classes appear in the code. Thus, we have the following: Controller controls application flow, managers keep needed persistence implementation, DAOs encapsulate JSON, CSV and API access specifics.

Similarly, we determined that the implementation specific for API must be kept within the API DAO. The manager may choose whether the implementation will be from the API or the local one, but the manager does not interact with ApiHelper directly. Only the APIDAO classes create and operate ApiHelper, hence the business managers are not bound to the implementation of how communication with the remote server takes place. From an implementation standpoint, it becomes clearer when represented in the UML diagram where the managers are dependent upon DAO interfaces/implementations, whereas APIDAO classes are dependent upon ApiHelper.

Dedication Estimate





Phase	Main activities	Jan Porcar	Francesc Mateu
Specification reading	Read and discuss requirements; identify new client/product types and persistence rules	2.5h	2.5h
Analysis & Design	Inheritance hierarchies, VAT hook pattern, DAO interfaces, UML class diagram iterations	8h	8h
Coding	New entity classes, DAO implementations, GeolocationApiManager integration, Controller flow	11h	13h
Documentation	JavaDoc for all new/modified classes, report writing, UML diagram finalisation	3h	1h
TOTAL		24.5h	24.5h

Understanding the Phase 3-4 specification consumed roughly 2 hours of time. This was because the specification had introduced various requirements that interacted together: various client types, their respective purchase restrictions, various product types and their respective prices, the option for cloud persistence, and the edge cases of VAT exempt products and unavailableness of service in certain cases. It was better to understand each requirement thoroughly before working with it to avoid the need to go backwards after working with something else.

Phase 4 analysis and design consumed 7 hours of time, most of which was spent thinking about the division of responsibility rather than drawing boxes in Astah. A big dilemma involved putting the VAT logic in SaleManager, as doing so would require type checkings whenever a new client type is added, resulting in the introduction of appliesVAT() method in the Client class. The problem of brand and model, which should originally be in the Product class but did not make sense in Service class, led to the introduction of the PhysicalProduct class as an intermediate level. Additionally, the definition of DAO classes as well as the discriminator used for the subtype determination from JSON was completed at this stage.

The coding process took about 12.5 hours and was the most challenging phase. Writing of the new entity classes itself was not difficult, however, implementing of DAOS was difficult due to the necessity of proper JSON deserialization of each product and client subtype, and implementing the API was difficult due to precise matching the required endpoint structure. Adding functionality of calculating the cost of products from an online shop using GeolocationApiManager also required working with some external API, possible exceptions and shutdowns, because of that it was crucial to try the API first, then validate locally stored files, and only after that shut down the application. It was important throughout all coding process to maintain backward compatibility with the Phase 1-2 implementation.

Documentation took about 2 hours. JavaDoc documentation was added to all classes created/modified, and at the same time this report was prepared along with reviewing of the UML diagram to ensure consistency between it and the code documentation.

Conclusions

Technical conclusions

One of the major lessons of this period is the importance that the early decisions made have on the difficulties of the following coding stage. For instance, choosing to implement VAT calculations in the client hierarchy and not within SaleManager ensured that the manager would not require any updates every time a new client type was implemented. While it is easy to underestimate such an approach while designing the architecture, it will become apparent later once you start adding more features into the application.

Finally, the dual persistence implementation was an example of what the use of the DAO pattern achieves in reality. Often the API was not available during development, but switching to local file storage simply through updating the objects passed to the controller and nothing else proved to save lots of time spent on debugging. The business layer indeed was not aware which backend worked at the moment – this is exactly what the pattern was designed for.

The checkout process was the one area where the most stuff could be potentially messed up at once. VATs, expiry discounts, pricing for services, shipping fees, inventory management, and sales tracking – everything had to happen in the right sequence to work perfectly in one transaction. With all of this business logic centralized within SaleManager, while the particular calculations were delegated to product and client classes, it became easier to think of the entire process at once rather than trying to wrap my head around individual types.

Personal conclusions

Prior to this assignment, I was already familiar with both inheritance and polymorphism well enough to describe them but had never used either of them in a system with concrete interacting objects. Having seen how the right application of an abstract method could reduce an entire class of conditionals in a manager class really showed me their value in practice.

Another lesson I learned is that although I knew all object-oriented design patterns by name, figuring out whether a particular one was applicable to any given situation is often the most difficult aspect of the process. In some cases, additional abstraction might have complicated the system needlessly, and recognizing when that happens comes through experience and is hard to learn in theory alone.

The final thing I have learned is that maintaining compatibility with previous versions of a piece of software requires planning for future changes. Although the requirement to maintain functionality from phases 1 and 2 at all times meant additional complexity, doing so has been useful because new functionalities always come into existence against the backdrop of existing code.

Bibliography

La Salle URL. (2025). *DPOO 2025–2026 project statement, API documentation, and submission regulations*. Course material.

Oracle Corporation. (2024). *Java SE 21 documentation*. Retrieved from https://docs.oracle.com/en/java/javase/21/

Google. (2024). *Gson user guide*. Retrieved from https://github.com/google/gson/blob/main/UserGuide.md

JetBrains. (2024). *IntelliJ IDEA documentation*. Retrieved from https://www.jetbrains.com/help/idea/

Astah. (2024). *Astah Professional UML documentation*. Retrieved from https://astah.net/support/